Big Data Analytics: Exam and CAT Cheat Sheet

MIT 91205 · Days 1 to 4 · Hadoop, Hive, Pig, Spark

1. Big Data basics

Big Data: data too big, too fast or too varied for one machine or a normal database to handle in acceptable time and cost.

V	Meaning	Example
Volume	size (TB to PB)	years of bank transactions
Velocity	speed of arrival	mobile money events per second
Variety	structured, semi, unstructured	tables, JSON, images
Veracity	quality, trust	missing or duplicate records
Value	useful insight	fraud caught, cost saved

Analytics types: descriptive (what happened), diagnostic (why), predictive (what will happen), prescriptive (what to do). Value and difficulty grow in that order.

Lifecycle: business understanding, acquisition, preparation (60 to 80% of effort), analysis, visualization, action and feedback. It loops.

Data types: structured (fixed schema), semi-structured (JSON, XML, logs), unstructured (text, audio, video).

2. Distributed systems

- **Scale up** (vertical): bigger machine, simple, hits a ceiling. **Scale out** (horizontal): more machines, cheap, needs coordination. Big Data scales out.
- **Characteristics**: resource sharing, concurrency, transparency, scalability, fault tolerance, heterogeneity.
- **Architectures**: client-server, peer-to-peer, master-worker (Hadoop, Spark).

CAP: during a network partition choose C or A

Type	Behaviour	Examples
CP	refuse some requests, never stale	HBase, MongoDB (majority)
AP	always answer, may be stale	Cassandra, DynamoDB
CA	only without partitions	single-node RDBMS

PACELC: if Partition, A or C; Else, Latency or Consistency.

ACID (RDBMS)	BASE (many NoSQL)
Atomic, Consistent, Isolated, Durable	Basically Available, Soft state, Eventual consistency
bank transfer	likes counter, social feed

Partitioning: range (good for range queries), hash (even load), list, round-robin. Risk: skew and hot spots.

Replication: master-slave, multi-master, quorum (W + R > N gives strong reads).

Failures: crash, omission, timing, Byzantine. Tools: replication, checkpointing, heartbeats, retries, speculative execution.

Processing: batch (bounded, throughput), stream (unbounded, latency), interactive (ad hoc queries).

Lambda = batch layer + speed layer + serving layer (two code paths). **Kappa** = one streaming path, replay the log to reprocess.

Layers: sources, ingestion, storage, processing, analytics, visualization; governance and security cut across all.

3. HDFS

NameNode (master)	DataNode (worker)
namespace, file to block to node map, in RAM; gets heartbeats and block reports; schedules re-replication	stores blocks on local disk, checksums, heartbeat every 3 s, serves reads and writes

- Block 128 MB (often 256 MB), replication 3. Rack-aware: replica 1 local, 2 and 3 on one other rack.
- **Read**: ask NameNode for locations, read directly from nearest DataNode. NameNode never carries data.
- **Write**: pipeline client to DN1 to DN2 to DN3, acks flow back.
- DataNode dead after about 10 min of silence; its blocks are copied from other replicas.
- **HA**: active + standby NameNode, shared edits via JournalNodes, ZooKeeper failover. **Secondary NameNode is not a backup**: it only merges fsimage and edits.
- **Federation**: several NameNodes, each owns part of the namespace.
- Good for: large files, write once read many, sequential scans. Bad for: millions of small files, low-latency random access, in-place edits.

```
blocks = ceil(file size / block size)
raw    = file size x replication
1 GB, 128 MB, R=3: 8 blocks, 24 replicas, 3 GB raw
```

4. YARN

Part	Job
ResourceManager	cluster-wide scheduler (FIFO, Capacity, Fair), starts each AM
NodeManager	one per node, launches containers, reports health
ApplicationMaster	one per app, asks RM for containers, runs and retries tasks
Container	bundle of memory + vCores on one node

HDFS manages **storage**; YARN manages **CPU and memory**. YARN lets MapReduce, Spark and Tez share one cluster.

5. MapReduce

```
input split -> map -> (combine) -> partition -> shuffle
& sort -> reduce -> HDFS
map:    (k1, v1)       -> list(k2, v2)
reduce: (k2, list(v2)) -> list(k3, v3)
```

- Split = logical unit of work (usually one block). One map task per split.
- Map output goes to **local disk**, not HDFS. Final output goes to HDFS, one `part-` file per reducer.
- **Partitioner**: `hash(key) mod R` picks the reducer. **Combiner**: local mini-reduce, may run 0, 1 or many times, only for associative and commutative ops.
- **Shuffle** is the most expensive phase: disk, network, sort.
- Data locality: run the map where the block is (node-local, then rack-local, then off-rack).
- Stragglers: speculative execution runs a copy; first to finish wins. Does not fix skew.
- Weak for: iterative jobs (each loop = new job + disk I/O), interactive queries.

Patterns

Pattern	Map emits	Reduce does
Word count	(word, 1)	sum
Inverted index	(word, docId)	dedupe list
Average	(key, (sum, count))	add, divide once
Reduce-side join	(key, tagged record)	match tags
Map-side join	small table in memory, no shuffle	none

> **Trap:** average is not a safe combiner (average of averages is wrong). Send (sum, count).

Amdahl: `S(N) = 1 / ((1 - p) + p / N)`. The serial part caps the speedup.

6. Hive

- SQL layer (HiveQL) over files in HDFS. Not a database, not an engine: runs on MapReduce, Tez or Spark.
- **Schema-on-read**: schema applied at query time, files untouched. Fast loading, errors show up late (as NULLs).

- **Metastore**: tables, columns, types, locations, partitions, SerDe. Backed by MySQL or PostgreSQL (Derby for tests only).
- Flow: client, HiveServer2, driver (parse, analyze, optimize, plan), engine, HDFS.

	Managed	External
Files owned by	Hive	someone else
DROP TABLE	deletes data + metadata	deletes metadata only

	Partitioning	Bucketing
Split by	column value, one folder each	hash(col) mod N files
Gives	partition pruning	sampling, bucket joins
Risk	too many tiny partitions	fixed count chosen early

```
CREATE TABLE sales (id BIGINT, amount DOUBLE)
PARTITIONED BY (dt STRING) STORED AS PARQUET;
SELECT region, SUM(amount) FROM sales
WHERE dt = '2024-03-15' GROUP BY region;  -- one folder read
```

Formats: text/CSV/JSON (easy, slow), Avro (row, schema evolution, pipelines), Parquet and ORC (columnar, compression, pushdown, analytics).

SerDe = serializer/deserializer: turns bytes into rows using the table schema.

UDF 1 row to 1 value, **UDAF** many rows to 1 value, **UDTF** 1 row to many rows (`explode`).

Order of evaluation: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT. WHERE filters rows, HAVING filters groups.

Trap: `col = NULL` is never true. Use `IS NULL`. `COUNT(*)` counts rows, `COUNT(col)` skips NULLs.

Trap: a function on the partition column (`year(dt) = 2024`) can stop pruning.

7. Pig

- Pig Latin: procedural **dataflow** scripts, each line names a new relation. Hive is declarative SQL.
- Types: **tuple** (ordered fields), **bag** (unordered tuples), **map** (key#value).
- Modes: local (`pig -x local`) and cluster. Nothing runs until `DUMP` or `STORE`.

```
s  = LOAD 'sales' USING PigStorage(',') AS (id:int,
region:chararray, amount:double);
g  = GROUP s BY region;
t  = FOREACH g GENERATE group, SUM(s.amount);
STORE t INTO 'out';
```

Joins: default (shuffle), `'replicated'` (small side in memory), `'merge'` (pre-sorted). Also SPLIT, UNION, ORDER, LIMIT, DESCRIBE, EXPLAIN.

Pick Hive when	Pick Pig when
SQL users, reports, shared tables, BI tools	multi-step ETL, nested data, branching flows

8. Spark architecture

- In-memory engine; not storage, not a resource manager. Runs on YARN, Kubernetes, standalone or local.
- **Driver**: runs main(), builds DAG, schedules. Dies = app fails. **Executor**: runs tasks, caches data. Dies = tasks rerun elsewhere.
- Client mode: driver on your machine (notebooks). Cluster mode: driver inside the cluster (production).
- **Application** > **job** (one per action) > **stage** (cut at each shuffle) > **task** (one per partition).
- **Lazy**: transformations only record the plan; an action runs it.
- **Lineage**: lost partitions are recomputed from their parents, no replication of intermediate data. `checkpoint()` cuts long lineage.

Narrow (no shuffle)	Wide (shuffle, new stage)
map, filter, flatMap, select, withColumn, union, coalesce	groupByKey, reduceByKey, join, distinct, repartition, orderBy

Transformations (lazy)	Actions (run a job)
map, filter, join, groupBy	count, collect, show, take, reduce, save/write

```
words.map(lambda w: (w, 1)).reduceByKey(add)  # combines before shuffle
words.map(lambda w: (w, 1)).groupByKey()      # ships every value: avoid
```

9. RDD, DataFrame, Dataset, SQL

	RDD	DataFrame	Dataset
Schema	no	yes	yes
Optimizer	none	Catalyst + Tungsten	Catalyst + Tungsten
Languages	all	all	Scala, Java
Use for	custom low-level logic	most work	typed JVM code

Spark SQL and DataFrames share one engine: same plan, same speed. Default choice: DataFrame or SQL.

Catalyst: parse, analyze (resolve names via catalog), optimize logical plan (pushdown, pruning, constant folding, join reorder), choose physical plan, generate code. **Tungsten**: binary rows, off-heap memory, whole-stage code generation.

10. Performance

- **Predicate pushdown**: filter at the source; Parquet skips row groups by min/max. **Partition pruning**: skip whole folders. **Projection pruning**: read only needed columns (no `SELECT *`).
- **Joins**: broadcast hash (small side < 10 MB default, no shuffle of big side), sort-merge (two big tables), shuffle hash.
- **Skew**: one task far slower than the rest. Fix: salting, AQE skew join, broadcast, handle hot keys apart.
- `repartition(n)` full shuffle, can grow. `coalesce(n)` merges, no full shuffle, use before writing.
- `spark.sql.shuffle.partitions` = 200 by default; tune to data size. AQE can coalesce at runtime.
- **Memory**: unified pool for execution (shuffle, join, sort) and storage (cache). Execution can evict cache.
- **Cache** only what is reused; `unpersist()` after. `cache()` = MEMORY_ONLY for RDD, MEMORY_AND_DISK for DataFrame.
- Avoid: `collect()` on big data, Python UDFs (use built-ins), thousands of tiny output files.
- Executors: about 4 to 5 cores each.

Reading `explain()`

`Exchange`	shuffle (stage boundary)
`BroadcastHashJoin`	small table copied to executors
`PushedFilters`	filter sent to file reader
`PartitionFilters`	folders skipped
`ReadSchema`	columns actually read
`InMemoryTableScan`	reading the cache

Spark UI (port 4040): Jobs, Stages (task time spread shows skew), Storage (cache), SQL (plan graph), Executors.

11. MLlib and GraphX

- **Transformer**: `transform()` (VectorAssembler, fitted model). **Estimator**: `fit()` returns a model (LogisticRegression, KMeans). **Pipeline**: ordered stages, reused for training and inference.
- Features: StringIndexer, OneHotEncoder, StandardScaler, Imputer, VectorAssembler.
- Algorithms: logistic regression, decision tree, random forest, k-means, ALS (recommendations).
- Metrics: classification (precision, recall, F1, AUC), regression (RMSE, MAE, R^2), clustering (silhouette).

- `spark.ml` (DataFrames) is current; `spark.mllib` (RDDs) is maintenance only.
- Small data that fits one machine: scikit-learn is usually faster.
- **GraphX**: vertex RDD + edge RDD. Pregel supersteps: get messages, update, send. PageRank, connected components, triangle counting. GraphFrames for Python.

12. Which tool?

Need	Use
store huge files reliably	HDFS or object store
one big nightly scan, legacy cluster	MapReduce or Hive
SQL reports over a data lake	Hive or Spark SQL
multi-step ETL scripts	Pig or Spark
iterative ML, graphs	Spark (MLlib, GraphX)
sub-second dashboards	precompute with Spark, serve from DuckDB, ClickHouse, Druid
real-time events	Kafka + Structured Streaming (Day 5)
data fits a laptop	pandas, DuckDB, scikit-learn

13. Ten traps examiners like

1. NameNode stores metadata, not data.
2. Secondary NameNode is not a hot standby.
3. Replication does not protect against deletes or bugs.
4. Schema-on-read still has a schema.
5. Hive is not a database and not always MapReduce.
6. Combiner may never run; result must not depend on it.
7. Spark spills to disk; it is not "all in memory".
8. Spark replaces MapReduce, not HDFS or YARN.
9. Caching data used once wastes memory.
10. Columnar is best for analytics, not for every workload.